Codificación de algoritmos

Breve descripción:

Este componente introduce los principios de la programación, los tipos de lenguajes y el uso de entornos de desarrollo. Aborda la sintaxis y estructuras de JavaScript, el manejo de datos, algoritmos básicos de búsqueda y ordenamiento, y el proceso de depuración, identificando errores de sintaxis, lógica y su adecuado tratamiento.

Octubre 2026

Tabla de contenido

Introducción 4

1. Fundamentos de programación 5

1.1. Conceptos básicos de programación 7

1.2. Tipos de lenguajes de programación: compilados e interpretados 14

2. Entornos de desarrollo y configuración del proyecto 18

2.1. Entornos de codificación 19

2.2. Instalación y configuración del entorno 20

2.3. Creación de proyectos de programación 22

3. Sintaxis y estructuras de programación en JavaScript 30

3.1. Tipos de datos, operadores y orden de evaluación 36

3.2. Expresiones, funciones y comentarios 45

3.3. Estructuras de selección 51

3.4. Estructuras de repetición (for y while) 55

3.5. Estructuras de salto (continue, break, return) 58

4. Estructuras de datos y algoritmos básicos 62

4.1. Estructuras de datos 65

4.2. Métodos de ordenamiento 69

4.3. Métodos de búsqueda para datos numéricos, textos y registros 73

5. Depuración y manejo de errores en programas 76

5.1. Depuración de código 77

5.2. Fallas de sintaxis 79

5.3. Fallas de lógica 86

5.4. Manejo de errores y excepciones 95

Síntesis 99

Glosario 101

Referencias bibliográficas 103

Créditos 104

Introducción

Los fundamentos de programación abarcan los principios básicos para desarrollar software, incluyendo conceptos como variables, algoritmos y control de flujo, así como la diferencia entre lenguajes compilados (que se traducen antes de ejecutarse) e interpretados (que se ejecutan línea por línea).

Adicionalmente, los entornos de desarrollo permiten escribir y organizar código mediante herramientas como editores o IDEs, junto con la instalación y configuración necesarias y la creación de proyectos; en JavaScript, la sintaxis define el uso de tipos de datos, operadores, expresiones, funciones y comentarios, así como estructuras de selección (if, switch), repetición (for, while) y salto (break, continue, return).

Finalmente, las estructuras de datos y algoritmos permiten organizar y procesar información mediante vectores, matrices, registros, métodos de ordenamiento y técnicas de búsqueda; finalmente, la depuración y manejo de errores incluye la identificación de fallas de sintaxis y lógica, así como el uso de mecanismos como try-catch para controlar excepciones durante la ejecución del programa.

1. Fundamentos de programación

Los fundamentos de programación constituyen la base esencial para el desarrollo de software o sistemas de información, permitiendo a los programadores comprender cómo diseñar, estructurar y ejecutar soluciones computacionales eficientes según la necesidad del cliente. Estos fundamentos incluyen conceptos clave como algoritmos, estructuras de control, tipos de datos y lógica de programación, los cuales son necesarios para resolver problemas de manera sistemática.

Uno de los elementos principales en la programación es el algoritmo, que se define como una serie de pasos ordenados y finitos que permiten resolver un problema específico. Los algoritmos deben ser claros, precisos y eficientes, ya que de ellos depende el rendimiento del programa.

En la programación también se utilizan variables, que son espacios en memoria donde se almacenan datos que pueden cambiar durante la ejecución del programa. Estas variables tienen tipos de datos específicos como números, cadenas de texto o valores booleanos.

Las estructuras de control permiten dirigir el flujo del programa. Entre ellas se encuentran:

- **Estructuras secuenciales**

Conjunto de instrucciones que se ejecutan de manera continua y ordenada, una después de la otra, siguiendo el flujo natural del programa.

- **Estructuras condicionales**

Permiten evaluar condiciones y tomar decisiones, ejecutando diferentes acciones según se cumpla o no una condición (if, else).

- **Estructuras repetitivas**

Facilitan la ejecución de un bloque de instrucciones varias veces, mientras se cumpla una condición o durante un número definido de iteraciones (for, while).

Otro aspecto importante es la sintaxis, que corresponde a las reglas que definen cómo se debe escribir el código en un lenguaje de programación. Una sintaxis incorrecta genera errores que impiden la ejecución del programa.

La lógica de programación es fundamental, ya que permite construir soluciones coherentes. Un programa puede estar bien escrito sintácticamente, pero si su lógica es incorrecta, no producirá los resultados esperados.

Los lenguajes de programación se clasifican en:

- **Compilados**

Necesitan que el código fuente sea traducido completamente a código máquina antes de ejecutarse. Este proceso genera un archivo ejecutable, mejora el rendimiento del programa y permite detectar muchos errores antes de la ejecución.

- **Interpretados**

Se ejecutan a través de un intérprete que traduce y ejecuta el código instrucción por instrucción en tiempo real. Esto facilita la prueba, la corrección de errores y la adaptación del programa a distintos entornos.

El desarrollo de habilidades en programación también implica el uso de buenas prácticas como la documentación del código, el uso de nombres descriptivos para variables y funciones, y la organización adecuada del proyecto.

Finalmente, comprender los fundamentos de programación es esencial para avanzar hacia temas más complejos como estructuras de datos, desarrollo web, inteligencia artificial y sistemas distribuidos.

1.1. Conceptos básicos de programación

Para desglosar un poco esta temática, se invita a acceder a al siguiente video introductorio, en el cual se presentan de manera general los conceptos básicos de la programación. A través de explicaciones claras y ejemplos sencillos, se aborda qué es un programa, cómo se estructuran los algoritmos y cuál es el papel de las variables y los datos, proporcionando una base conceptual para el estudio de la programación:

Video 1. Conceptos básicos de programación



Enlace de reproducción del video

Transcripción del video: Conceptos básicos de programación

Un programa es un conjunto de instrucciones que una computadora ejecuta para realizar una tarea específica. Estas instrucciones se escriben en un lenguaje de programación, el cual actúa como intermediario entre el ser humano y la máquina, permitiendo transformar ideas y soluciones en acciones que el sistema puede procesar de manera automática.

Uno de los elementos esenciales de la programación es el algoritmo, definido como una secuencia ordenada y lógica de pasos que permiten resolver un problema o alcanzar un objetivo concreto. Los algoritmos están presentes en la vida cotidiana y constituyen la base de cualquier solución informática. Todo algoritmo sigue una estructura básica compuesta por las siguientes etapas:

a) Inicio
b) Datos de entrada
c) Proceso
d) Datos de salida
e) Fin

Este ejemplo representa un algoritmo sencillo que permite calcular la ganancia obtenida a partir de una compra y una venta. El algoritmo inicia solicitando dos datos de entrada: el precio de compra y el precio de venta. Luego, durante el proceso, se realiza una operación aritmética restando el precio de venta al precio de compra para obtener la ganancia. Finalmente, el resultado calculado se muestra como dato de salida y el algoritmo finaliza su ejecución.

Transcripción del video: Conceptos básicos de programación
Recuerde: un buen algoritmo debe ser claro, preciso y eficiente, de forma que pueda comprenderse fácilmente y ejecutarse sin errores. Además, las variables representan espacios de memoria donde se almacenan datos que pueden cambiar durante la ejecución del programa. Estos datos poseen un tipo definido, como entero, decimal, texto o booleano, lo cual permite que el programa interprete y procese la información de manera adecuada.

Ejemplos de tipos de datos que existen en lenguajes de programación son:

Tabla 1. Tipos de datos

Tipo de dato	Representación	Tamaño (bytes)	Rango de valores	Valor por defecto	Clase asociada
byte	Numérico entero con signo.	1	-128 a 127	0	byte
short	Numérico entero con signo.	2	-32768 a 32767	0	short
int	Numérico entero con signo.	4	-2147483648 a 2147483647	0	integer
long	Numérico entero con signo.	8	-9223372036854775808 a 9223372036854775807	0	long
float	Numérico en coma flotante (simple, IEEE 754).	4	±3.4×10^-38 a ±3.4×10^38	0.0	float

Tipo de dato	Representación	Tamaño (bytes)	Rango de valores	Valor por defecto	Clase asociada
double	Numérico en coma flotante (doble, IEEE 754).	8	±1.8×10^-308 a ±1.8×10^308	0.0	double
char	Carácter Unicode.	2	\u0000 a \uFFFF	\u0000	character
boolean	Dato lógico.	-	true o false	false	boolean
void	-	-	-	-	void

Los operadores permiten realizar operaciones sobre los datos, existen operadores aritméticos, relacionales y lógicos que permiten construir expresiones más complejas.

Estos son algunos ejemplos:

- **Operadores aritméticos**

Se utilizan para realizar operaciones matemáticas básicas.

Suma (+) → 5 + 3 = 8.

Resta (−) → 5 − 3 = 2.

Multiplicación (*) → 5 * 3 = 15.

División (/) → 6 / 2 = 3.

Módulo (%) → 5 % 2 = 1.

- **Operadores relacionales**

Sirven para comparar dos valores y devuelven un resultado lógico (verdadero o falso).

Mayor que (>) → 10 > 5 → verdadero.

Menor que (<) → 3 < 2 → falso.

Igual (==) → 4 == 4 → verdadero.

Diferente (!=) → 4 != 5 → verdadero.

Mayor o igual (>=) → 5 >= 5 → verdadero.

- **Operadores lógicos**

Permiten combinar o negar condiciones.

AND (&&) → (5 > 3 && 2 < 4) → verdadero.

OR (||) → (5 < 3 || 2 < 4) → verdadero.

NOT (!) → !(5 > 3) → falso.

Por su parte, las estructuras de control son fundamentales para dirigir el flujo del programa. Estas incluyen estructuras condicionales como:

a) If:

1. INICIO
2. Leer numero
3. SI numero > 0 ENTONCES
4. Mostrar "El número es positivo"
5. FIN SI

6. FIN

b) For:

1. INICIO
2. PARA i ← 1 HASTA 5 HACER
3. Mostrar i
4. PARA
5. FIN

Muestra los números del 1 al 5

c) While:

1. INICIO
2. Leer numero
3. MIENTRAS numero < 0 HACER
4. Mostrar "Ingrese un número positivo"
5. Leer numero
6. FIN MIENTRAS
7. Mostrar "Número válido"
8. FIN

Además, la programación requiere seguir buenas prácticas como la correcta indentación del código, el uso de nombres significativos y la documentación mediante comentarios.

Finalmente, comprender estos conceptos básicos es esencial para avanzar hacia temas más complejos en ingeniería de software. Es importante tener en cuenta los elementos de la programación:

- **Algoritmos**: pasos para resolver problemas.
- **Variables**: almacenamiento de datos.
- **Tipos de datos**: clasificación de la información.
- **Operadores**: manipulación de datos.
- **Estructuras de control**: flujo del programa.
- **Funciones**: reutilización de código.

El aprendizaje de la programación también implica desarrollar habilidades de pensamiento lógico y resolución de problemas.

Se debe tener presente:

- Practicar constantemente para afianzar los conocimientos adquiridos.
- El uso de diagramas de flujo puede ayudar a visualizar la lógica de un algoritmo antes de implementarlo.
- Los errores son parte del proceso de aprendizaje y permiten mejorar las habilidades del programador.
- El trabajo en equipo y el uso de herramientas colaborativas son fundamentales en proyectos de software.
- Comprender estos conceptos facilita el aprendizaje de cualquier lenguaje de programación.
- La abstracción permite simplificar problemas complejos en partes manejables.
- La programación es una habilidad clave en la transformación digital actual.
- El uso de estándares mejora la calidad del código.
- La revisión de código permite detectar errores y mejorar soluciones.

- El aprendizaje de la programación también implica desarrollar habilidades de pensamiento lógico y resolución de problemas.

1.2. Tipos de lenguajes de programación: compilados e interpretados

En el ámbito de la ingeniería de software, los lenguajes de programación se clasifican comúnmente según la forma en que su código es ejecutado. Dos de las categorías más importantes son los lenguajes compilados e interpretados. Esta clasificación influye directamente en el rendimiento, portabilidad, seguridad y proceso de desarrollo del software.

- **Lenguajes compilados**

Este tipo de lenguaje es aquel en el que el programa escrito por el desarrollador (código fuente) se traduce completamente a lenguaje máquina antes de ejecutarse, mediante una herramienta llamada compilador. Este proceso genera un archivo ejecutable que puede correr directamente en el sistema sin necesidad del código original.

A diferencia de otros tipos de lenguajes, en los compilados la traducción ocurre una sola vez antes de la ejecución, lo que permite que el programa funcione de manera más rápida y eficiente, ya que el computador entiende directamente las instrucciones generadas.

Sus características son:

- El código fuente se traduce completamente a lenguaje máquina antes de ejecutarse.

- Se genera un archivo ejecutable independiente del código original.
- Ofrecen mayor rendimiento y eficiencia en tiempo de ejecución.
- Detectan muchos errores en tiempo de compilación.

Ejemplos: C, C++, Go, Rust.

El proceso de compilación transforma el código escrito por el programador en instrucciones que el procesador puede ejecutar directamente. Esto permite que los programas sean rápidos y eficientes, especialmente en sistemas donde el rendimiento es crítico.

- **Lenguajes interpretados**

Un lenguaje interpretado es aquel en el que el código fuente se ejecuta línea por línea en tiempo real, mediante un programa llamado intérprete, sin necesidad de generar previamente un archivo ejecutable.

En este tipo de lenguajes, el código no se traduce completamente antes de ejecutarse, sino que el intérprete va leyendo, traduciendo y ejecutando cada instrucción al momento. Esto permite una mayor flexibilidad y facilita la prueba y corrección de errores durante el desarrollo.

Por ejemplo, en lenguajes como JavaScript o Python, se puede escribir una instrucción y ejecutarla inmediatamente sin necesidad de compilar todo el programa.

Sus características son:

- El código se ejecuta línea por línea mediante un intérprete.
- No se genera un archivo ejecutable independiente.
- Son más flexibles y fáciles de depurar.

- Permiten mayor portabilidad entre sistemas.

Ejemplos: Python, JavaScript, Ruby.

En los lenguajes interpretados, el intérprete analiza y ejecuta el código en tiempo real. Esto facilita la experimentación y el desarrollo rápido, aunque puede implicar menor rendimiento en comparación con los compilados.

La siguiente tabla, relaciona un poco cada proceso que diferencia cada uno de estos lenguajes:

Tabla 2. Comparación de lenguajes de programación

Característica	Compilados	Interpretados	Impacto
Ejecución.	Previo a ejecución	En tiempo real	Velocidad vs. flexibilidad.
Rendimiento.	Alto	Medio	Optimización vs facilidad.
Errores.	En compilación	En ejecución	Detección temprana vs. dinámica.
Portabilidad.	Dependiente del sistema	Alta	Compatibilidad.
Ejemplos.	C, C++	Python, JS	Aplicación práctica.

La elección entre un lenguaje compilado o interpretado depende del contexto del proyecto, incluyendo requisitos de rendimiento y velocidad de desarrollo; además, se debe tener en cuenta lo siguiente:

- En sistemas embebidos y videojuegos, los lenguajes compilados suelen ser preferidos por su eficiencia.

- En aplicaciones web y análisis de datos, los lenguajes interpretados ofrecen mayor productividad.
- El uso de entornos de ejecución como máquinas virtuales permite mejorar la portabilidad del software.
- El proceso de desarrollo también cambia: los lenguajes interpretados permiten pruebas rápidas sin necesidad de compilación.

Los compiladores modernos también incluyen herramientas de análisis estático para mejorar la calidad del código, comprender estas diferencias permite seleccionar la mejor tecnología para cada tipo de solución.

La elección del lenguaje impacta la arquitectura del sistema, los tiempos de entrega y el mantenimiento del software, por lo que es fundamental analizar las características técnicas antes de decidir.

2. Entornos de desarrollo y configuración del proyecto

En la ingeniería de software, los entornos de trabajo y la correcta configuración de un proyecto son fundamentales para crear aplicaciones robustas, mantenibles y escalables. Estas plataformas no solo facilitan la escritura de código, sino que incorporan herramientas para depurar, probar, administrar dependencias y gestionar versiones. Una configuración adecuada asegura el funcionamiento coherente de los componentes en contextos como prueba, implementación y producción.

Un entorno de desarrollo es el conjunto de herramientas que permiten a los programadores crear software de forma eficiente. Estos entornos pueden variar desde editores simples hasta plataformas altamente integradas; encontrando:

- **IDE (Entorno de Desarrollo Integrado)**

Incluyen herramientas completas como depuradores, compiladores y gestión de proyectos (IntelliJ, Eclipse, Visual Studio).

- **Editores de código**

Son ligeros y extensibles mediante plugins (Visual Studio Code, Sublime Text).

- **Entornos en la nube**

Permiten programar desde cualquier lugar sin instalar software (GitHub Codespaces, Replit).

- **Terminales y CLI**

Interfaces basadas en comandos para automatización y control avanzado.

- **Herramientas de depuración**

Permiten identificar errores paso a paso durante la ejecución.

2.1. Entornos de codificación

Constituyen un componente esencial en el desarrollo de software moderno. Proporcionan un conjunto de herramientas que permiten a los desarrolladores escribir, analizar, depurar y mantener código de forma eficiente. La elección adecuada de un entorno impacta directamente en la productividad, la calidad del software y la colaboración en equipos de desarrollo.

Dentro de sus funcionalidades clave se destacan:

- Resaltado de sintaxis que mejora la legibilidad.
- Autocompletado inteligente que reduce errores.
- Depuración paso a paso.
- Integración con control de versiones.

En cuanto al soporte para extensiones, estas funcionalidades permiten optimizar el proceso de desarrollo, reducir errores y mejorar la comprensión del código. A continuación, una breve relación de esta codificación:

Tabla 3. Entornos de codificación

Tipo	Ejemplo	Ventajas	Limitaciones
Editor.	VS Code.	Ligero, rápido.	Configurar extensiones.
IDE.	IntelliJ.	Completo.	Mayor consumo.

Tipo	Ejemplo	Ventajas	Limitaciones
Nube.	Replit.	Accesible.	Depende de internet.

Sus buenas prácticas en el uso de entornos se dan al momento de:

- Seleccionar el entorno según el tipo de proyecto.
- Mantener herramientas actualizadas.
- Utilizar control de versiones.
- Evitar instalar extensiones innecesarias.
- Documentar la configuración.

Los entornos de codificación influyen directamente en la productividad del desarrollador. Un entorno bien configurado facilita la detección de errores, mejora la organización del código y permite una integración eficiente con otras herramientas del ciclo de desarrollo.

2.2. Instalación y configuración del entorno

JavaScript es uno de los lenguajes más utilizados en el desarrollo web moderno. Para trabajar de manera eficiente, es fundamental contar con un entorno de codificación correctamente instalado y configurado. Los siguientes son unos pasos necesarios para preparar un entorno profesional de desarrollo en JavaScript:

a) Instalación de herramientas básicas

- Instalar Node.js: esto permite ejecutar JavaScript fuera del navegador.
- Instalar npm (gestor de paquetes incluido con Node.js).

- Instalar un editor de código como Visual Studio Code.
- Instalar Git para control de versiones.

Node.js es esencial porque proporciona el entorno de ejecución para JavaScript en el servidor. Además, npm permite instalar librerías necesarias para el desarrollo.

Visual Studio Code

Como apoyo a este contenido, el siguiente video como INSTALAR JAVA en visual studio code (2025) 🚀 muestra paso a paso cómo instalar y configurar Java en Visual Studio Code, explicando la preparación del entorno de trabajo y la integración de las herramientas necesarias para comenzar a programar desde el editor.

b) Configuración del entorno

- Verificar instalación con comandos: node -v y npm -v.
- Configurar variables de entorno si es necesario.
- Instalar extensiones en VS Code (ESLint, Prettier, Live Server).
- Configurar formato automático de código.
- Inicializar proyecto con npm init.

c) Buenas prácticas en JavaScript

- Usar ESLint para detectar errores.
- Formatear código con Prettier.
- Usar control de versiones con Git.
- Organizar el proyecto correctamente.
- Actualizar dependencias regularmente.

Un entorno de JavaScript bien configurado mejora la productividad, permitiendo a los desarrolladores trabajar de manera más eficiente y profesional.

Como base de apoyo a los procesos de instalación de JavaScript se presentan las siguientes guías:

- **JavaScript Windows**

Este documento ¿Cómo puedo descargar e instalar Java en un equipo con Windows de forma manual? presenta una guía sencilla para realizar la descarga manual e instalación de Java en Windows, explicando los pasos básicos para obtener el instalador oficial, ejecutarlo correctamente y verificar que la instalación se haya completado con éxito.

- **JavaScript macOS**

Este documento Instalación y uso de Oracle Java en macOS relaciona una guía de ayuda sobre instalación y uso de Java en macOS, donde se explica cómo obtener Java para Mac, los requisitos del sistema, las instrucciones para instalarlo y actualizarlo, así como otros aspectos como activar o desinstalar Java y cómo comprobar la versión instalada, todo desde la documentación oficial de Java.com.

2.3. Creación de proyectos de programación

La creación de proyectos requiere de la identificación de requerimientos o requisitos de un sistema de información del cual se describen los servicios que ha de ofrecer, las restricciones asociadas a su funcionamiento; es decir, las propiedades o restricciones que deben estar registradas, determinadas de forma precisa.

Los siguientes son tres aspectos informativos a tener en cuenta:

- **Definición general del software**

Consiste en describir de forma clara el sistema de información o la solución a desarrollar, explicando la idea central, la funcionalidad principal y el alcance del proyecto. Incluye los propósitos, objetivos generales y el contexto en el que será utilizado, permitiendo comprender su utilidad y valor dentro del entorno para el cual fue concebido.

- **Especificación de requerimientos**

Es fundamental detallar de manera precisa los requerimientos técnicos y funcionales del sistema, así como los alcances, restricciones y limitaciones de la implementación. Debe indicarse si el proyecto forma parte de un sistema previamente existente y, en tal caso, aclarar si corresponde a una nueva versión, una ampliación o una derivación, garantizando una correcta integración.

- **Procedimientos de instalación y prueba**

Debe organizarse de forma detallada el proceso para obtener, instalar y ejecutar el software, incluyendo los pasos necesarios para su correcta puesta en funcionamiento. También se deben describir las pruebas básicas de verificación, las condiciones del entorno, la plataforma requerida y los criterios que aseguran que el sistema opere de manera adecuada.

A continuación, se presenta una descripción integrada de los elementos clave que conforman la documentación de un proyecto de software, abordando de manera progresiva su definición, desarrollo, implementación y validación:

- **Definición del proyecto de software**

En primer lugar, en la definición del proyecto de software se debe describir de forma clara la idea general, explicando la funcionalidad principal del sistema de información y el problema que busca resolver. Asimismo, es necesario establecer los objetivos del desarrollo, indicando la necesidad que cubre el sistema, e identificar a los usuarios o entidades que interactuarán con él, junto con el nivel de experiencia del público al que está dirigido el informe.

- **Especificación de requerimientos**

A partir de esta base, la especificación de requerimientos debe detallar las pautas generales que rigen el funcionamiento del proyecto y los requisitos funcionales que describen los servicios, tareas y comportamientos que el sistema debe realizar. De igual manera, se debe aclarar la autoría del software, indicando si se trata de un desarrollo original o si forma parte de un sistema preexistente, además de definir los alcances y limitaciones del sistema en coherencia con los objetivos establecidos.

- **Procedimientos de desarrollo**

Posteriormente, en los procedimientos de desarrollo del sistema de información se describen las herramientas y tecnologías empleadas, tales como entornos de desarrollo integrados, plataformas y lenguajes de programación necesarios para la implementación. Junto con ello, se presenta una planificación general que expone la metodología utilizada y los principales pasos seguidos durante la ejecución del proyecto.

- **Procedimientos de instalación y prueba**

Finalmente, los procedimientos de instalación y prueba deben contemplar los requisitos no funcionales que afectan el desempeño del sistema, como restricciones relacionadas con tiempos de respuesta. Además, se debe incluir una guía sencilla para la obtención e instalación del software, dirigida al nivel de usuario definido previamente, así como las especificaciones técnicas de la plataforma y los entornos requeridos para la prueba y ejecución del sistema.

A. Arquitectura del sistema

Un software de tamaño pequeño se identifica de la composición de varios módulos o partes interconectados de alguna forma. La especificación de la arquitectura del sistema de información de los cuáles son estas partes; qué rol tienen dentro del software y la forma en que se organizan e interconectan.

La información sobre la arquitectura debe incluir como mínimo:

- **Descripción jerárquica**

Indica de qué forma se organizan jerárquicamente los componentes del sistema de información; es decir, anotar si los mismos están organizados en paquetes, espacios de nombres o bien si el software posee una estructura monolítica.

- **Diagrama de módulos**

Consiste en el diseño un diagrama donde se representan todas las partes que componen el sistema; es decir, qué tipos interacciones intervienen y las relaciones que existen entre estas. Su objetivo consiste en presentar una perspectiva generalizada de

la arquitectura y los componentes del sistema, no deberá contener detalles técnicos sobre los módulos o las conexiones entre estos.

- **Descripción individual de los módulos**

Cada módulo o parte del sistema, requiere una breve descripción del mismo, la cual debería incluir mínimamente cada proceso o tarea que desarrolla:

  - **Descripción y propósito**: ¿qué es y qué debería hacer el módulo?
  - **Responsabilidad y restricciones**: ¿cuál es su función dentro del sistema?, ¿qué tareas puede y no puede hacer?
  - **Dependencias**: describir cuales son los requisitos de cada módulo, es decir se debe contestar a preguntas tales como, ¿qué necesita o requiere el módulo para funcionar?, ¿necesita de servicios brindados por otros módulos o por librerías externas?
  - **Implementación**: describir en qué archivo o archivos se encuentra la implementación del módulo.

No es el objetivo de esta sección dar detalles de cómo se realiza la implementación de los módulos, sino únicamente dar una idea generalizada de la existencia de los módulos que hacen parte del sistema.

B. Diseño del modelo de datos

Es donde se deben identificar cuáles son los agentes involucrados en el sistema de información y nombrarlas en un formato de lenguaje agnóstico. El siguiente es uno ejemplo de este lenguaje:

- **Descripción**

El sistema debe calcular el total de una compra multiplicando el precio de un producto por la cantidad seleccionada.

- **Pseudocódigo**

INICIO

LEER precio

LEER cantidad

Total= precio * cantidad

MOSTRAR total

FIN

Puede ser un diseño orientado a objetos, relacional, etc., se debe tener una idea general del módulo de datos: entidades, atributos y las relaciones entre ellas. Par consiguiente, es imprescindible incluir diagramas o gráficos que ayuden a visualizar y entender el modelo.

Un programa, aplicación o librería puede a su vez trabajar con varios tipos de datos (entrada, internos o de salida).

C. Descripción de procesos y servicios ofrecidos por el sistema

Se debe explicar de manera clara cuáles son las tareas o servicios que el sistema ofrece y los procesos que ejecuta para cumplirlos, con el fin de que el lector comprenda su funcionamiento general y la forma en que pueden utilizarse. Para apoyar esta explicación, es conveniente emplear recursos como pseudo-algoritmos o diagramas de

flujo que representen el comportamiento del sistema. No se espera mostrar el código ni detallar funciones específicas, sino describir brevemente qué hace cada proceso, su propósito y los datos de entrada y salida, indicando su tipo, cantidad y significado.

En este punto es fundamental que el código fuente de la aplicación esté adecuadamente documentado mediante comentarios claros y pertinentes. Estos constituyen la base de la documentación del proyecto y deben permitir comprender, a un nivel general, el funcionamiento de los procesos y servicios del sistema, así como la estructura y propósito de sus funciones, subrutinas, módulos o clases.

D. Documentación técnica - Especificación API

Se indica el propósito y breve descripción de cada método/función, con su prototipo indicando argumentos (nombre, tipo, propósito de cada uno) y respuesta (tipo, descripción).

Para llevar a cabo esta tarea, es posible utilizar una variedad de herramientas de generación de documentación automática, a partir del código en el encabezado de cada función (Javadoc, PHPDoc, Doxygen, etc).

La documentación técnica debe pensarse como el manual del programador y apuntar a aquellas personas que estarán a cargo de mantener, ampliar o crear un proyecto derivado a partir de nuestro proyecto.

Para finalizar, se dan los siguientes aspectos relevantes que se deben tener presentes:

- **Guía de uso del programa**

Indicar de forma clara cómo ejecutar el programa, especificando los parámetros obligatorios y opcionales, la función de cada uno y el comportamiento por defecto cuando no se proporcionan. Esta información constituye la base del manual de usuario final de la aplicación.

- **Descripción de la solución y arquitectura**

Incorporar diagramas de flujo y explicaciones a nivel de método que describan la estrategia general de resolución del problema. Debe mostrarse cómo se organiza la solución y cómo interactúan los distintos módulos entre sí, permitiendo comprender la lógica global del sistema sin necesidad de revisar el código.

- **Documentación de tipos de datos abstractos**

Los tipos de datos abstractos deben estar correctamente documentados en el código y descritos en el manual, indicando cómo se representa cada estructura, cuáles son sus limitaciones y qué métodos ofrece para la manipulación y gestión de los datos, facilitando su uso y mantenimiento.

- **Conclusiones y lecciones aprendidas**

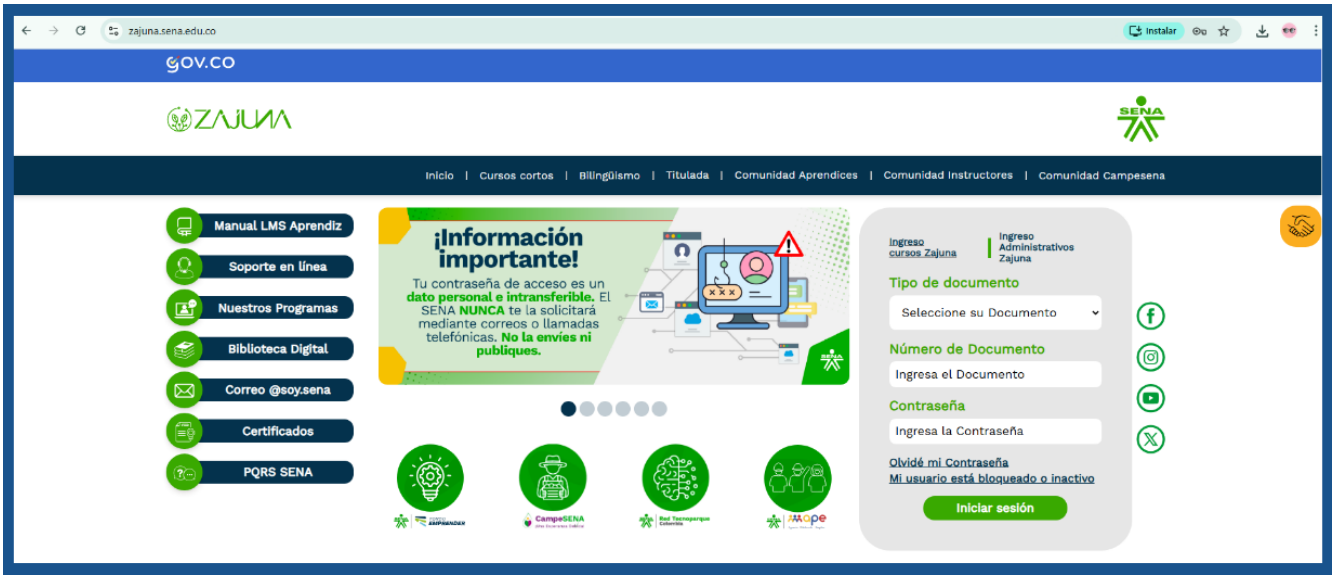
Incluir una sección final donde se sinteticen las principales dificultades encontradas durante el desarrollo, las decisiones o políticas adoptadas para resolverlas, las restricciones del problema original, los casos particulares abordados y los aprendizajes obtenidos a partir de la experiencia del proyecto.

3. Sintaxis y estructuras de programación en JavaScript

JavaScript es un lenguaje de programación que mejora HTML al agregar animaciones, interactividad y efectos visuales dinámicos. Permite una retroalimentación automática en las páginas web; es decir que todos códigos que son agregados a una página web se reflejan inmediatamente. JavaScript puede crear interfaces interactivas, como presentaciones animadas y consejos emergentes, adicionalmente es considerado como el lenguaje fundamental en el desarrollo de software actual, especialmente en aplicaciones web. Comprender su sintaxis y estructuras de programación es esencial para construir soluciones eficientes, mantenibles y escalables.

La siguiente imagen ejemplifica este proceso:

Figura 1. Interactividad Zajuna



Un ejemplo concreto es la página Zajuna del SENA, donde gran parte de la interactividad se soporta en JavaScript. Gracias a ello, es posible encontrar efectos dinámicos como el resaltado de textos, sombras, animaciones y movimiento de imágenes, entre otros elementos que mejoran la experiencia del usuario.

Zajuna es una plataforma web interactiva que:

- Permite iniciar sesión.
- Permite gestionar cursos, usuarios y contenido.
- Navegar entre cursos.
- Mostrar contenido dinámico.

Todo eso requiere JavaScript para funcionar correctamente.

Adicionalmente, JavaScript trasciende el ámbito de las páginas web, ya que puede emplearse en aplicaciones como Yahoo! Widgets, Google Apps y en la automatización de tareas dentro de software de Adobe, entre otros usos. Asimismo, ha ganado relevancia en el desarrollo del lado del servidor mediante plataformas como Node.js, lo que amplía las oportunidades para el desarrollo full-stack utilizando un solo lenguaje.

Con JavaScript se puede crear:

- Páginas web.
- Aplicaciones móviles.
- Sistemas completos (frontend + backend).
- Juegos.
- Programas de escritorio.

Este conocimiento permite abordar proyectos de software de forma estructurada y eficiente; además, su comprensión facilita la resolución de problemas complejos en entornos reales de desarrollo. En el contexto actual del desarrollo de software, estas habilidades son altamente valoradas y demandadas en la industria tecnológica.

- **Estructura de programación en JavaScript**

Para entender esta estructura, se relaciona el siguiente ejemplo de un código de un programa en Java, seguido de su debida explicación:

MiClase.java

```
1    class MiClase {
2    public static void main(String[] args){
3    System.out.println(“Hola mundo”);
4    } // Fin del método main
5    } // Fin de la clase MiClase
```

Hola mundo

Nota: escriba el código del ejemplo anterior en un editor de texto plano, guarde el archivo con el nombre y extensión.

MiClase.java

Es importante respetar las letras en mayúsculas y las letras en minúsculas; es decir, el nombre del archivo debe ser exactamente el que se muestra en la línea 1, así también, la extensión debe ser .java. Como cualquier nuevo conocimiento, conforme se

realicen más ejercicios, poco a poco se irá familiarizando con la estructura general de los programas en Java.

A continuación, se da la explicación de cada línea del código:

- **Línea 1: class MiClase {**

Marca el inicio de la definición de la clase MiClase. En Java, todo programa debe contar con al menos una clase. Para declarar dicha clase se utiliza la palabra reservada class, seguida del nombre que identificará la clase. Las palabras reservadas siempre se escriben en minúsculas y son exclusivas del lenguaje Java. Finalmente, la llave de apertura { indica que, a partir de ese punto, todo el contenido pertenece a esa clase.

- **Línea 2: public static void main(String[] args){**

Corresponde al método que se ejecuta para iniciar el programa. Por ahora no es necesario comprender completamente esta línea; a medida que se avance en el aprendizaje, se entenderá el significado de cada una de sus partes. Lo importante es reconocer que todo programa en Java necesita una clase, y dentro de ella debe existir un método main para poder ejecutarse. Al final de la línea se observa nuevamente una llave de apertura {, la cual señala el comienzo de las instrucciones que ejecutará el programa.

- **Línea 3: System.out.println(“Hola mundo”);**

Se encarga de mostrar el texto Hola mundo en la pantalla. Es importante notar que, al ejecutar el programa, el mensaje aparece sin las comillas dobles. En Java, las cadenas de texto se escriben entre comillas dobles. Aunque aún no se comprenda cada parte de esta instrucción, con el tiempo se logrará entender en detalle. Por ahora, si se

desea imprimir texto en pantalla, basta con utilizar la instrucción mostrada en esta línea.

- **Línea 4: } // Fin del método main**

Inicia con una llave de cierre }, lo cual indica el término de las instrucciones que forman parte del método main. Al utilizar dos barras diagonales //, todo el contenido que sigue en esa misma línea es ignorado por el compilador de Java. Esto se conoce como comentario de línea y se utiliza para añadir aclaraciones o información que facilite la comprensión del código.

- **Línea 5: } // Fin de la clase MiClase**

Comienza también con una llave de cierre }, indicando el final del cuerpo de la clase MiClase. Esta línea incluye un comentario de línea, por lo que todo el texto que aparece después de las dos barras diagonales // será ignorado por el compilador hasta que se llegue a la siguiente línea. Todos los programas en Java seguirán esta misma estructura.

Por otra parte, los símbolos en Java son importantes, pues son la estructura del programa como tal y para que su ejecución se dé, se tiene lo siguiente:

- **Los paréntesis ()**

Los paréntesis cumplen dos funciones principales en un programa Java, las cuales son:

  - Delimitan listas de parámetros en los métodos.
  - Alteran la prioridad en una expresión aritmética.

- **Las llaves {}**

Las llaves en el lenguaje de programación Java se utilizan en los siguientes casos:

  - Definen un bloque de código, el cual puede estar anidado.
  - Permiten declarar valores iniciales de los arreglos, separados por comas.

- **Los corchetes []**

Los corchetes se emplean en un programa Java para:

  - Declarar arreglos (vectores).
  - Acceder a los elementos de los arreglos.

- **El punto y coma «;»**

El punto y coma indica el final de una instrucción en un programa Java.

  - Finaliza una línea de código.

- **La coma «,»**

La coma se utiliza para:

  - Separar identificadores en declaraciones.

- **El punto «.»**

El punto en un programa Java se utiliza para:

  - Acceder a métodos de una clase instanciada.
  - Referenciar subpaquetes dentro de un paquete.

3.1. Tipos de datos, operadores y orden de evaluación

Este apartado introduce los conceptos fundamentales relacionados con los tipos de datos, los operadores y las reglas que determinan el orden en que se evalúan las expresiones, elementos esenciales para comprender cómo se procesan y ejecutan las instrucciones dentro de un programa.

- **Tipos de datos**

Los tipos de datos permiten establecer cómo se representan y gestionan los valores dentro de un programa, facilitando la organización de la información y el correcto procesamiento de las operaciones durante la ejecución del software. Dentro de ellos se encuentran:

a) Tipos de datos primitivos

Los tipos de datos utilizados en el lenguaje de programación Java, conocidos como tipos de datos primitivos, incluyen para números enteros: byte, short, int y long; para números reales: float y double; para valores lógicos: boolean; y para caracteres: char. El tamaño de estos tipos de datos primitivos está definido por la plataforma Java y se clasifican de la siguiente manera:

Tabla 4. Clasificación tipos de datos primitivos

Tipo	Primitiva	Definición
Entero.	byte	Entero en complemento a dos.
-	short	con signo de 8 bits.
-	int	con signo de 16 bits.
-	long	con signo de 32 bits.

Tipo	Primitiva	Definición
Real.	float	con signo de 64 bits.
-	double	la norma IEEE 754 de 32 bits.

Es posible encapsular los tipos de datos primitivos para manejarlos como cualquier otro objeto en Java. Por ejemplo, existe una clase envolvente para el tipo primitivo int llamada integer. La utilidad de estas clases envolventes se comprenderá mejor al estudiar las clases contenedoras de datos.

Ejemplos de declaración de variables:

La declaración de variables en un programa Java se realiza utilizando el siguiente formato: tipo variables;

int a; (declara la variable entera a).

int a, b; (declara las variables enteras a y b).

int a, b, c; (declara las variables enteras a, b y c).

int a=0; (declara la variable entera y la inicializa con valor cero).

float a=0.0f, b=5.0f; (declara las variables reales a y b y las inicializa a la variable a en 0.0 y a la variable b en 5.0).

Nota: en el caso de solo las variables float, cuando se inicializa la variable con un valor real este valor debe terminar en f de float.

doublé a=0.0, b=0.0; (declara las variables reales a y b y las inicializa con valor cero, no requiere de ninguna letra en el valor cero).

boolean sw; (declara la variable lógica sw).

char letra = ‘a’; (declara la variable char letra y la inicializa con el valor de a minúscula, observe que el valor a està entre apóstrofes).

b) Tipos de datos referencia

En Java, los objetos (instancias de clases) se gestionan mediante referencias. Cuando se crea una nueva instancia de una clase utilizando el operador new, este retorna una referencia del tipo de la clase. Para comprenderlo mejor, se relaciona un ejemplo:

Se tiene definida e implementada una clase llamada factorial, como se muestra a continuación:

```
public class Factorial {
// Atributos
    int n, fact;
   // Metodos
   public void EstablecerNumero(int n1)
   {
      n= n1;
   }
   public void CalcularFactorial()
   {
```

```
fact = 1;
      for (int i = 1; i <= n; i++)
{
fact = fact * i;
}
   }
   public int ObtenerFactorial()
{
return fact;
}
} // Fin de la clase Factoria
```

Y en la clase de aplicación, se tiene el siguiente código de programa:

```
import java.util.Scanner; public class EjecutaFactorial
{
public static void main(String[] args)
{
// atributos
      int n;
      // establecer objeto de entrada
```

```
        Scanner teclado = new Scanner(System.in);
        // Instancia de clase
        Factorial objFactorial = new Factorial();
        System.out.print ("Ingrese Numero; ");
        n=teclado.NextInt();
        objFactorial.EstablecerNumero(n);
        objFactorial.CalcularFactorial();
        System.out.println("***** RESULTADO *******");
    System.out.println("El factorial es: "+objFactorial.ObtenerFactorial());


    }
    }
```

- **Operadores Java**

Los operadores permiten realizar asignaciones, cálculos, comparaciones y evaluaciones lógicas dentro de un programa; a continuación, se explican y ejemplifican para mayor entendimiento, algunos de los más utilizados en el lenguaje Java:

a) Operadores de asignación Java operador

= para Asignación simple,

ejemplo:

C = 8;

operador += para Asignación suma, ejemplo:

n+=5;

n=n+5;

Estos dos ejemplos dan el mismo resultado operador -= para Asignación Resta,

ejemplo:

n-=5;

n=n-5;

Estos dos ejemplos dan el mismo resultado:

operador *= para Asignación Multiplicación, ejemplo:

n*=5;

n=n*5;

Estos dos ejemplos dan el mismo resultado operador /= para Asignación División, ejemplo:

n/=5;

n= n/5;

Estos dos ejemplos dan el mismo resultado operador % = para Asignación Módulo, ejemplo:

n%=5;

n=n%5;

Estos dos ejemplos dan el mismo resultado

b) Operadores aritméticos Java

operador + para Suma, ejemplo c=a+b

operador - para Resta, ejemplo c=a-b

operador * para Multiplicación, ejemplo c=a*b

operador / para División, ejemplo c=a/b

operador % para Modulo (resto), ejemplo c=a%b

operador ++ para Incrementar, ejemplo a++ ; significa a=a+1 operador -- para Decrementar, ejemplo a--; significa a=a-1

c) Operadores relacionales Java

operador < para Menor a operador > para Mayor a

operador <= para Menor o igual a

operador >= para Mayor o igual a

operador == para Igual a

operador != para Diferente a

d) Operadores lógicos Java

operador ! para No (Complemento

operador && para y (And)}

operador || para o (or)

- **Orden de evaluación**

En JavaScript se refiere a la forma en que el intérprete determina qué partes de una expresión se ejecutan primero y en qué secuencia. Esto es importante porque puede influir directamente en el resultado de una operación. Dentro de ellos se encuentran:

a) Precedencia de operadores

JavaScript utiliza reglas de precedencia para definir qué operadores se evalúan primero.

Ejemplo:

let resultado = 2 + 3 * 4;

Primero se realiza la multiplicación (3 * 4 = 12), luego la suma (2 + 12 = 14).

b) Asociatividad

Cuando los operadores tienen la misma precedencia, se aplica la asociatividad.

Ejemplo:

let resultado = 10 - 5 - 2;

Se evalúa de izquierda a derecha: (10 - 5) = 5, luego 5 - 2 = 3.

c) Uso de paréntesis

Los paréntesis permiten modificar el orden de evaluación.

Ejemplo:

let resultado = (2 + 3) * 4;

Primero se resuelve el paréntesis (2 + 3 = 5) y luego la multiplicación (5 * 4 = 20).

d) Evaluación de izquierda a derecha

En general, las expresiones se evalúan de izquierda a derecha respetando la precedencia.

Ejemplo:

console.log(1 + 2 * 3);

e) Evaluación corta (Short-circuit)

Algunos operadores lógicos detienen la evaluación anticipadamente.

Ejemplo:

false && console.log('Hola');

El mensaje no se ejecuta porque el resultado ya es false.

En conclusión, orden de evaluación depende de:

- La precedencia de operadores.
- La asociatividad.
- El uso de paréntesis.
- La evaluación lógica (short-circuit).

3.2. Expresiones, funciones y comentarios

Este apartado aborda los elementos básicos que permiten construir instrucciones significativas en JavaScript, organizar la lógica del programa y documentar el código, facilitando su comprensión, reutilización y mantenimiento dentro del proceso de desarrollo de software.

- **Expresiones:**

En JavaScript, una expresión es cualquier fragmento de código que produce un valor. Las expresiones son la base de la programación, ya que permiten realizar cálculos, comparaciones, asignaciones y evaluaciones lógicas dentro de un programa.

Los siguientes, son los tipos de expresiones:

- **Expresiones aritméticas:** se utilizan para realizar operaciones matemáticas como suma, resta, multiplicación y división.

  Ejemplo:

  let resultado = 5 + 3 * 2; // Resultado: 11

- **Expresiones relacionales:** sirven para comparar valores y devuelven un resultado booleano (true o false).

  Ejemplo:

  let esMayor = 10 > 5; // true

- **Expresiones lógicas:** permiten combinar condiciones utilizando operadores como AND (&&), OR (||) y NOT (!).

  Ejemplo:

let valido = (10 > 5) && (3 < 8); // true

- **Expresiones de asignación:** se usan para asignar valores a variables.

  Ejemplo:

  let x = 10;

- **Expresiones de cadena:** permiten trabajar con texto (strings).

  Ejemplo:

  let saludo = 'Hola ' + 'mundo';

- **Expresiones de función:** son funciones definidas dentro de una expresión.

  Ejemplo:

  const suma = function(a, b) { return a + b; };

Las expresiones en JavaScript se evalúan siguiendo reglas de precedencia, asociatividad y uso de paréntesis.

Ejemplo:

let resultado = (2 + 3) * 4; // 20

Los siguientes son algunos ejercicios prácticos:

- Ejercicio 1

  let a = 10 + 5 * 2; // ¿Cuál es el resultado?

- Ejercicio 2

let b = (10 + 5) * 2; // ¿Cuál es el resultado?

- Ejercicio 3

  let c = (5 > 3) && (2 < 1); // ¿Cuál es el resultado?

- **Funciones**

En JavaScript, una función es un bloque de código diseñado para realizar una tarea específica. Las funciones permiten reutilizar código, organizar mejor los programas y facilitar el mantenimiento.

Una función se define utilizando la palabra reservada function, seguida de un nombre, paréntesis y un bloque de código.

Ejemplo:

```
function saludar() {
  console.log('Hola mundo');
}
```

Las siguientes son las funciones más destacadas y sus ejemplificaciones:

- **Funciones con parámetros:** las funciones pueden recibir datos de entrada llamados parámetros.

  Ejemplo:

```
function sumar(a, b) {
  return a + b;
}
```

- **Funciones con retorno:** una función puede devolver un valor utilizando la palabra return.

  Ejemplo:

  ```
  let resultado = sumar(5, 3); // 8
  ```

- **Funciones anónimas:** son funciones sin nombre que pueden asignarse a una variable.

  Ejemplo:

  ```
  const saludo = function() {
    console.log('Hola');
  };
  ```

- **Funciones flecha (Arrow Functions):** son una forma más corta de escribir funciones.

  Ejemplo:

  ```
  const suma = (a, b) => a + b;
  ```

- **Ámbito (Scope) de las funciones:** el scope define el alcance de las variables dentro de una función.

  Ejemplo:

  ```
  function ejemplo() {
    let x = 10;
  }
  ```

```
// x no es accesible fuera de la función
```

Igualmente, se presentan los siguientes ejercicios prácticos:

- **Ejercicio 1**: crear una función que reciba un número y devuelva su cuadrado.
- **Ejercicio 2**: crear una función que reciba un nombre y muestre un saludo.
- **Ejercicio 3**: crear una función flecha que multiplique dos números.

Comprendiendo lo que tiene que ver tanto con las expresiones, como con las funciones, se relaciona la importancia de cada una:

- **Importancia de las funciones**

Las funciones son fundamentales en JavaScript porque permiten dividir el código en partes más pequeñas, reutilizables y fáciles de entender, comprender el uso de funciones en JavaScript es esencial para desarrollar aplicaciones eficientes y bien estructuradas.

- **Importancia de las expresiones**

Las expresiones permiten construir la lógica de un programa, realizar cálculos, evaluar condiciones y manipular datos.

Comprender las expresiones en JavaScript es fundamental para desarrollar programas eficientes. Estas permiten realizar múltiples operaciones y son esenciales en cualquier aplicación.

- **Comentarios**

Los comentarios en el lenguaje de programación Java se utilizan para añadir anotaciones o mensajes aclaratorios dentro del código; no son ejecutados y se clasifican en tres tipos:

- **Comentarios de una sola línea**

  Inician con los caracteres // seguidos del contenido del comentario.

  Ejemplo:

  // Atributos int a,b,c; // a,b,c representan atributos enteros de la clase

- **Comentarios de bloques**

  Comienzan con los caracteres /* en cualquier parte de la línea de código, continúan con el contenido del comentario y finalizan con los caracteres /

  Ejemplo:

  /

  Texto de los comentarios en bloque

  / comentarios en bloque

  /

  xxxxxxxxxxxxxxxxxx

  */

- **Comentarios de documentación**

  Se utilizan para documentar el programa. Empiezan con los caracteres /** en cualquier parte de la línea de instrucción, luego se escribe el contenido del comentario y finalizan con los caracteres /

  Ejemplo:

  /*

  Texto de los comentarios de documentación

  /

  Este es un ejemplo de comentarios de documentación:

  /* * *

  @author jorger

  */

3.3. Estructuras de selección

Las estructuras de selección en JavaScript permiten tomar decisiones dentro de un programa, ejecutando diferentes bloques de código según se cumplan o no determinadas condiciones. Las siguientes son este tipo de estructuras:

- **Estructura if**

Se utiliza para ejecutar un bloque de código si una condición es verdadera.

Ejemplo:

```
if (x > 0) {
  console.log('Número positivo');
}
```

- **Estructura if - else**

Permite ejecutar un bloque de código si la condición es verdadera y otro si es falsa.

Ejemplo:

```
if (x > 0) {
  console.log('Positivo');
} else {
  console.log('Negativo o cero');
}
```

- **Estructura if - else if - else**

Se utiliza para evaluar múltiples condiciones.

Ejemplo:

```
if (x > 0) {
  console.log('Positivo');
} else if (x < 0) {
```

```
  console.log('Negativo');
} else {
  console.log('Cero');
}
```

- **Estructura switch**

Permite seleccionar una opción entre múltiples casos.

Ejemplo:

```
switch (dia) {
  case 1:
    console.log('Lunes');
    break;
  case 2:
    console.log('Martes');
    break;
  default:
    console.log('Otro día');
}
```

- **Operador ternario**

Es una forma abreviada de la estructura if-else.

Ejemplo:

```
let mensaje = (x > 0) ? 'Positivo' : 'Negativo';
```

Ejercicios propuestos:

a) Crear un programa que determine si un número es mayor de edad.

b) Crear un programa que determine el mayor de dos números.

c) Usar switch para mostrar el nombre de un mes según su número.

Soluciones sugeridas:

a)

```
if (edad >= 18) {
  console.log('Mayor de edad');
} else {
  console.log('Menor de edad');
}
```

b)

```
if (a > b) {
  console.log(a);
} else {
  console.log(b);
}
```

c)

```
switch (mes) {
 case 1: console.log('Enero'); break;
```

```
 case 2: console.log('Febrero'); break;

 default: console.log('Otro');

}
```

Las estructuras de selección son fundamentales en JavaScript, ya que permiten controlar el flujo del programa y tomar decisiones basadas en condiciones.

3.4. Estructuras de repetición (for y while)

Las estructuras de repetición permiten ejecutar un conjunto de instrucciones de manera repetida, ya sea un número determinado de veces o mientras se cumpla una condición específica. En Java, los bucles for y while son ampliamente utilizados para automatizar tareas repetitivas y controlar el flujo del programa de forma eficiente, según las necesidades de cada situación.

- **Repetición sobre un rango determinado: for**

El for es una estructura de repetición que permite ejecutar un conjunto de instrucciones tantas veces como sea necesario. Esta estructura consta de tres partes fundamentales: la variable de inicio, la condición y el incremento.

Bucle for, su sintaxis y formato es la siguiente:

```
for(iniciación; condición; incremento)

{

// Bloque de instrucciones

}
```

En la sección de iniciación se puede declarar una variable de control del ciclo, cuyo alcance estará limitado al propio bucle. Tanto en la parte de iniciación como en la de incremento se pueden incluir varias expresiones separadas por comas, pero esto no es permitido en la parte de la condición.

La condición debe ser una expresión booleana o una expresión que se evalúe como un valor booleano.

Ejemplos:

```
for (int i=0;i<=5;i++){

/* variable inicial = i, condición i menor o igual a 5, incremento i++ equivalente a i+1 */

System.out.println("hola");

}
```

Como resultado, este bucle for imprimirá 6 veces la palabra hola, ya que en Java el ciclo for inicia desde el valor definido (en este caso 0). Por lo tanto, si se requieren 10 repeticiones, la condición debería ser i<=9, comenzando desde cero.

```
for (int j = 1; j< 4;j++){

System.out.print(objArray01.muestra(j)+" ");

}
```

- **Repeticiones condicionales: while**

El while, al igual que el for, es una estructura de repetición; la diferencia principal es que el while no requiere una variable de control ni un incremento explícito, ya que

solo necesita una condición para ejecutarse. El while evalúa dicha condición y, si se cumple, ejecuta el bloque de instrucciones; posteriormente vuelve a evaluar la condición. Cuando esta deja de cumplirse, el ciclo se detiene y el programa continúa con su flujo normal.

Su sintaxis y funcionamiento son similares a los de C/C++. En la estructura while, la condición se evalúa antes de ejecutar el bloque de instrucciones.

```
while(condición) {
// Bloque de instrucciones
}
```

Ejemplo

```
int A=0;
int B=5;
while(A < B) { // repetir mientras A sea menor que B //
A=A+1;
}
```

Este ejemplo del ciclo while finalizará cuando A deje de ser menor que B; es decir, cuando la variable A alcance el valor de 6. En cada iteración del ciclo, A se incrementa en 1; por lo tanto, cuando A sea igual a 6, la condición ya no se cumple y el ciclo termina.

3.5. Estructuras de salto (continue, break, return)

Las estructuras de salto en JavaScript permiten alterar el flujo normal de ejecución de un programa. Estas estructuras son útiles para controlar ciclos, salir de bloques de código o devolver valores desde funciones.

- **Sentencia continue**

La sentencia continue se utiliza para omitir la iteración actual de un bucle y continuar con la siguiente.

Ejemplo:

```
for (let i = 0; i < 5; i++) {
  if (i === 2) {
    continue;
  }
  console.log(i);
}
// Imprime: 0, 1, 3, 4
```

- **Sentencia break**

La sentencia break se utiliza para terminar de forma inmediata un bucle o una estructura switch.

Ejemplo:

```
for (let i = 0; i < 5; i++) {
```

```
  if (i === 3) {
    break;
  }
  console.log(i);
}
// Imprime: 0, 1, 2
```

- **Sentencia return**

La sentencia return se utiliza dentro de una función para devolver un valor y finalizar su ejecución.

Ejemplo:

```
function sumar(a, b) {
  return a + b;
}
let resultado = sumar(3, 4);
// resultado = 7
```

Las diferencias principales entre estas sentencias son:

- **Break** : finaliza completamente un ciclo o switch.
- **continue**: salta a la siguiente iteración del ciclo.
- **return**: finaliza una función y devuelve un valor.

Ejercicios propuestos:

- Crear un ciclo que imprima números del 1 al 10, pero que se detenga cuando llegue a 6.

```
for (let i = 1; i <= 10; i++) {
  if (i === 6) break;
  console.log(i);
}
```

- Crear un ciclo que imprima números del 1 al 10, pero que omita el número 5.

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) continue;
  console.log(i);
}
```

- Crear una función que reciba un número y retorne si es par o impar.

```
function esPar(n) {
  return (n % 2 === 0) ? 'Par' : 'Impar';
}
```

Las estructuras de salto son herramientas clave en JavaScript que permiten controlar el flujo de ejecución de manera eficiente, facilitando la construcción de programas más claros y optimizados.

4. Estructuras de datos y algoritmos básicos

Las estructuras de datos y los algoritmos son conceptos fundamentales en programación, ya que permiten gestionar y procesar la información de forma eficiente para resolver problemas. Las estructuras de datos definen cómo se disponen y administran los datos dentro de un programa, facilitando su acceso y uso adecuado.

A continuación, se presentan algunas de las principales estructuras de datos utilizadas en programación, con una breve descripción y ejemplos sencillos que permitirán comprender su funcionamiento y aplicación práctica en la gestión de la información:

- **Arreglos (arrays)**

Permiten almacenar múltiples valores en una sola variable.

Ejemplo:

```
let numeros = [1, 2, 3, 4];
```

- **Objetos (objects)**

Permiten almacenar información en pares clave–valor.

Ejemplo:

```
let persona = {
 nombre: 'Juan',
 edad: 25
};
```

- **Pilas (stacks)**

Siguen el principio LIFO (último en entrar, primero en salir).

Ejemplo:

```
let pila = [];

pila.push(1);

pila.pop();
```

- **Colas (queues)**

Siguen el principio FIFO (primero en entrar, primero en salir).

Ejemplo:

```
let cola = [];

cola.push(1);

cola.shift();
```

De igual manera, se presentan algunos algoritmos básicos ampliamente utilizados en programación, orientados a la búsqueda, organización y recorrido de datos, con ejemplos que facilitan la comprensión de su lógica y aplicación práctica:

- **Búsqueda**

Permite encontrar un elemento dentro de una lista.

Ejemplo:

```
function buscar(lista, valor) {
```

```
 for (let i = 0; i < lista.length; i++) {
  if (lista[i] === valor) return i;
 }
 return -1;
}
```

- **Ordenamiento**

Permite organizar los datos.

Ejemplo:

```
function ordenar(lista) {
 for (let i = 0; i < lista.length; i++) {
  for (let j = 0; j < lista.length - 1; j++) {
   if (lista[j] > lista[j + 1]) {
    let temp = lista[j];
    lista[j] = lista[j + 1];
    lista[j + 1] = temp;
   }
  }
 }
 return lista;
```

}

- **Recorrido**

Consiste en visitar cada elemento de una estructura.

Ejemplo:

```
let numeros = [1,2,3];

numeros.forEach(n => console.log(n));
```

Las estructuras de datos y algoritmos permiten organizar mejor la información, optimizar el rendimiento de los programas y resolver problemas de manera eficiente, comprender las estructuras de datos y los algoritmos es esencial para desarrollar software eficiente y de calidad en JavaScript.

4.1. Estructuras de datos

Generalmente, al desarrollar programas, se declaran e inicializan variables para realizar diferentes operaciones. Estas resultan muy útiles para almacenar de manera temporal valores que serán utilizados durante la ejecución del programa. En esta sección se estudian los arreglos, los cuales ofrecen la posibilidad de contar con un contenedor de valores u objetos de un mismo tipo. Son especialmente útiles cuando se requiere almacenar un conjunto de datos homogéneos, como por ejemplo calificaciones, nombres de personas o temperaturas; los ejemplos pueden ser muy diversos. Sin duda, en muchos casos, el uso de arreglos facilitará las operaciones en los programas que se desarrollen.

Ejemplo:

String[] arregloTextos = new String[7];

Para definir un arreglo, se indica el tipo de dato que contendrá. Luego, se agregan corchetes de apertura y cierre [ ].

En el anterior ejemplo, se utiliza String[] como tipo de arreglo. Posteriormente, se escribe el nombre con el cual se identificará el arreglo. Se añade el operador de asignación =, seguido de la palabra reservada new, y nuevamente el tipo de dato con sus corchetes, pero esta vez indicando dentro de ellos la longitud del arreglo.

Para este caso, new String[7].

Un aspecto importante a considerar es que los arreglos pueden ser de cualquier tipo, incluyendo datos primitivos.

Ejemplo de Arreglos en java:

```
public class EjemploArreglo {
    public static void main(String[] args) {
        // Declaración e inicialización de un arreglo
        int[] numeros = {10, 20, 30, 40, 50};
        // Acceso a un elemento del arreglo
        System.out.println("Primer elemento: " + numeros[0]);
        // Modificar un elemento
        numeros[2] = 35;
```

```
        // Recorrer el arreglo con un ciclo for
        System.out.println("Elementos del arreglo:");
        for (int i = 0; i < numeros.length; i++) {
            System.out.println(numeros[i]);
        }
    }
}
```

Explicación:

- int[] numeros → Declara un arreglo de enteros.
- {10, 20, 30, 40, 50} → Inicializa el arreglo con valores.
- numeros[0] → Accede al primer elemento (los índices empiezan en 0).
- numeros.length → Obtiene el tamaño del arreglo.
- El for permite recorrer todos los elementos.

- **Matrices**

En JavaScript, las matrices son una estructura de datos bidimensional que se implementan como un arreglo de arreglos. Esto permite organizar la información en forma de filas y columnas, similar a una tabla.

Ejemplo:

```
let matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

Partiendo de lo anterior, se describen los aspectos fundamentales para trabajar con matrices en JavaScript, desde el acceso y recorrido de sus elementos hasta sus aplicaciones más comunes y algunas buenas prácticas que facilitan un uso correcto y eficiente de esta estructura de datos:

- **Acceso a elementos**

  Para acceder a un elemento se utilizan dos índices: fila y columna.

  Ejemplo:

  ```
  matriz[0][1]; // Accede al valor 2
  ```

- **Recorrido de una matriz**

  Para recorrer una matriz se utilizan ciclos anidados.

  Ejemplo:

  ```
  for (let i = 0; i < matriz.length; i++) {
    for (let j = 0; j < matriz[i].length; j++) {
      console.log(matriz[i][j]);
  ```

```
 }
}
```

- **Modificación de valores**

Los valores de una matriz pueden modificarse accediendo a su posición.

Ejemplo:

```
matriz[1][2] = 10;
```

- **Aplicaciones prácticas**

Las matrices se utilizan en múltiples escenarios como:

  - Representación de tablas de datos.
  - Juegos (tableros).
  - Procesamiento de imágenes.
  - Sistemas de coordenadas.

- **Buenas prácticas**
  - Mantener filas del mismo tamaño.
  - Validar índices antes de acceder.
  - Usar nombres descriptivos.
  - Evitar estructuras desordenadas.

4.2. Métodos de ordenamiento

Los métodos de ordenamiento son algoritmos fundamentales en programación que permiten organizar los elementos de un arreglo en un orden específico, ya sea

ascendente o descendente. En el desarrollo de software, el ordenamiento es clave para optimizar búsquedas, mejorar el rendimiento y facilitar el análisis de datos.

Los algoritmos de ordenamiento se utilizan en múltiples escenarios reales, como, por ejemplo:

- Listas de usuarios.
- Sistemas de reportes.
- Procesamiento de datos.
- Bases de datos.
- Aplicaciones web y móviles.

Un buen algoritmo de ordenamiento puede reducir significativamente el tiempo de ejecución de un programa.

a) Tipos de algoritmos de ordenamiento

Existen dos grandes categorías:

- Algoritmos simples ($O(n^2)$): fáciles de entender, pero menos eficientes.
- Algoritmos eficientes ($O(n \log n)$): usados en aplicaciones reales.

b) Bubble Sort (Ordenamiento burbuja)

Este algoritmo compara elementos adyacentes e intercambia sus posiciones si están en el orden incorrecto.

Complejidad: $O(n^2)$.

Ejemplo:

```
function burbuja(arr){
```

```
for(let i=0;i<arr.length;i++){
 for(let j=0;j<arr.length-1;j++){
  if(arr[j]>arr[j+1]){
   let temp=arr[j];
   arr[j]=arr[j+1];
   arr[j+1]=temp;
  }
 }
}
return arr;
}
```

Estos algoritmos se dividen de la siguiente manera y se explican sus respectivas acciones:

- **Selection Sort**

  Busca el valor mínimo del arreglo y lo coloca en la posición correcta.

  Complejidad: $O(n^2)$.

  Uso: útil para aprendizaje y estructuras pequeñas.

- **Insertion Sort**

  Inserta cada elemento en su posición adecuada dentro de una lista parcialmente ordenada.

  Complejidad: $O(n^2)$.

  Ventaja: eficiente en listas pequeñas o casi ordenadas.

- **QuickSort**

  Algoritmo basado en la técnica “divide y vencerá”. Utiliza un pivote para dividir el arreglo.

  Complejidad promedio: O(n log n).

  Es uno de los algoritmos más utilizados en sistemas reales.

- **MergeSort**

  Divide el arreglo en partes iguales, las ordena y luego las combina.

  Complejidad: O(n log n)

  Ventaja: rendimiento estable incluso con grandes volúmenes de datos.

  Basado en lo anterior, se resume y relaciona la comparación de algoritmos:

- **Burbuja**

  $O(n^2)$ - simple.

- **Selección**

  $O(n^2)$ - básico.

- **Inserción**

  O(n^2) - eficiente en datos pequeños.

- **QuickSort**

  O(n log n) - rápido en la práctica.

- **MergeSort**

  O(n log n) - estable y eficiente.

c) Método sort() en JavaScript

JavaScript incluye un método nativo para ordenar arreglos:

array.sort((a,b)=>a-b);

Este método utiliza algoritmos optimizados internamente, por lo que es el más recomendado en la mayoría de los casos.

4.3. Métodos de búsqueda para datos numéricos, textos y registros

En el desarrollo de software, uno de los problemas más frecuentes consiste en localizar información dentro de un conjunto de datos. Este proceso se conoce como búsqueda y es fundamental en prácticamente cualquier aplicación informática, desde sistemas simples hasta plataformas complejas que manejan grandes volúmenes de información.

En JavaScript, los métodos de búsqueda permiten encontrar elementos específicos dentro de estructuras de datos, como arreglos u objetos. Estos elementos pueden ser de diferentes tipos, tales como datos numéricos, cadenas de texto o

registros más complejos (por ejemplo, objetos que representan usuarios, productos o transacciones).

La importancia de los métodos de búsqueda radica en que permiten acceder de manera eficiente a la información, lo cual impacta directamente en el rendimiento y la experiencia del usuario. Por ejemplo, en una aplicación web, cada vez que un usuario busca un producto, filtra resultados o accede a un registro específico, se está utilizando algún tipo de algoritmo o método de búsqueda.

Existen diferentes enfoques para realizar búsquedas, los cuales varían según la estructura de los datos y el contexto del problema. Entre los más comunes se encuentran:

- Búsqueda lineal, que recorre todos los elementos hasta encontrar el valor deseado.
- Búsqueda binaria, que es más eficiente, pero requiere que los datos estén previamente ordenados.
- Métodos nativos de JavaScript como find(), filter(), includes() e indexOf(), que facilitan la búsqueda sin necesidad de implementar algoritmos desde cero.

Cuando se trabaja con datos numéricos, la búsqueda suele centrarse en encontrar valores específicos o rangos dentro de listas. En el caso de datos de texto, es común buscar coincidencias exactas o parciales dentro de cadenas. Por otro lado, cuando se manejan registros (objetos), la búsqueda puede implicar condiciones más complejas, como encontrar un objeto que cumpla ciertos criterios en una o varias de sus propiedades.

Además, en el contexto profesional, no solo es importante encontrar los datos, sino hacerlo de manera eficiente y escalable. A medida que el volumen de información crece, elegir el método de búsqueda adecuado se vuelve crucial para evitar problemas de rendimiento.

En JavaScript moderno, el uso de funciones de orden superior y métodos de los arreglos ha simplificado considerablemente la implementación de algoritmos de búsqueda, permitiendo escribir código más limpio, legible y mantenible. No obstante, comprender los principios que los sustentan sigue siendo fundamental para tomar decisiones informadas y optimizar soluciones en escenarios de mayor complejidad.

En conclusión, los métodos de búsqueda en JavaScript constituyen una herramienta fundamental para el manejo de datos, ya que permiten localizar, filtrar y procesar información de manera eficiente, siendo una base clave en el desarrollo de aplicaciones modernas.

5. Depuración y manejo de errores en programas

En el desarrollo de aplicaciones, los errores y las excepciones son inevitables. Al tener el rol de programadores, se posee la responsabilidad manejarlos adecuadamente para evitar que la experiencia del usuario se vea afectada. Una correcta gestión de errores también permite a los desarrolladores depurar el código y comprender mejor sus causas para poder solucionarlos de manera eficiente.

JavaScript ha sido un lenguaje de programación ampliamente utilizado durante más de tres décadas. Con él se desarrollan aplicaciones web, móviles, PWA y del lado del servidor, utilizando diversas bibliotecas populares (como ReactJS) y frameworks (como Next.js, Remix, entre otros).

Al ser un lenguaje de tipado dinámico, JavaScript presenta el desafío de manejar correctamente la seguridad de tipos. TypeScript ayuda a controlar este aspecto, pero aun así es necesario gestionar los errores en tiempo de ejecución de manera eficiente dentro del código.

Errores como TypeError, RangeError y ReferenceError probablemente sean familiares si se tiene experiencia desarrollando en JavaScript. Todos estos pueden generar datos inválidos, comportamientos incorrectos en la navegación, resultados inesperados o incluso provocar que la aplicación falle completamente, lo cual afecta negativamente la experiencia del usuario final.

5.1. Depuración de código

La depuración de código (debugging) es el proceso mediante el cual los desarrolladores identifican, analizan y corrigen errores dentro de un programa. En JavaScript, esta práctica es fundamental debido a que muchos errores se detectan en tiempo de ejecución.

¿Qué es la depuración?

La depuración consiste en detectar errores, analizar el comportamiento del programa y corregir fallos en la lógica del código.

Las siguiente son las herramientas de depuración:

- **Consola (console.log)**

Permite mostrar información en la consola.

Ejemplo:

```
let nombre = 'Carlos';

console.log(nombre);
```

Resultado: Carlos

- **Herramientas de desarrollador (DevTools)**

Permiten inspeccionar, depurar y analizar el comportamiento del código directamente en el navegador.

Ejemplo:

```
let edad = 18;
```

```
let resultado = edad >= 18 ? "Mayor de edad" : "Menor de edad";
```

- **Breakpoints**

Permiten pausar la ejecución del programa en una línea específica.

Ejemplo:

```
for (let i = 0; i < 5; i++) {
  // breakpoint aquí
}
```

- **Sentencia debugger**

Detiene la ejecución del código desde el propio programa.

Ejemplo:

```
function prueba() {
  debugger;
  let x = 10;
}
```

- **Visual Studio Code**

Editor de código que permite depuración avanzada.

Ejemplo:

```
function suma(a, b) {
  return a + b;
```

```
}

console.log(suma(2,3));
```

Igualmente, para detectar errores en Java existen unas técnicas de depuración como son:

- Ejecución paso a paso.
- Inspección de variables.
- Análisis del flujo del programa.
- Uso de registros (logs).

5.2. Fallas de sintaxis

En el desarrollo de software, uno de los errores más comunes que enfrentan los programadores son las fallas de sintaxis, también conocidas como SyntaxError. Estas ocurren cuando el código escrito no cumple con las reglas gramaticales del lenguaje de programación, impidiendo que el intérprete o el motor de JavaScript pueda comprender y ejecutar correctamente el programa.

JavaScript, al ser un lenguaje interpretado, analiza el código antes de ejecutarlo. Durante este proceso, si detecta una estructura incorrecta — como paréntesis sin cerrar, comillas mal utilizadas, palabras reservadas mal escritas o una mala organización del código — genera un error de sintaxis y detiene la ejecución del programa. Esto significa que, a diferencia de otros errores, las fallas de sintaxis deben corregirse obligatoriamente antes de que el código pueda ejecutarse.

Las fallas de sintaxis pueden presentarse en diferentes formas, tales como:

- Falta de punto y coma al finalizar instrucciones.
- Uso incorrecto de llaves {} o paréntesis ().
- Declaraciones incompletas o mal estructuradas.
- Errores en cadenas de texto (comillas abiertas o mal cerradas).
- Uso incorrecto de palabras reservadas del lenguaje.

Para la sintaxis de fallas, se ejemplifica con los siguientes códigos de programación:

- try {

  // Código que puede generar un error

  Ejecuta el bloque de código y verifica si se produce algún error.

- } catch (error) {

  // Código para manejar el error

  Se ejecuta cuando ocurre un error en el bloque try, capturando el objeto de error.

- } finally {

  // Código que siempre se ejecuta, ocurra o no un error

  }

  Ejecuta el código independientemente de si ocurrió o no un error (es opcional).

A continuación, se presentan distintas formas y escenarios prácticos para generar, gestionar y clasificar errores, desde la creación de errores personalizados hasta

el uso de estructuras modernas para el control de fallos y la identificación de los tipos de errores más comunes en JavaScript:

a) Generar errores personalizados

Es posible generar errores de forma manual utilizando la instrucción throw.

```
function checkAge(age) {
if (age < 18) {
throw new Error("You must be 18 or older.");
}
return "Access granted";
}
try {
console.log(checkAge(16));
} catch (error) {
console.log("Error:", error.message);
}
```

¿Cuándo usar el manejo de errores de JavaScript?

El manejo de errores en JavaScript es fundamental en situaciones donde pueden presentarse problemas inesperados. Algunos casos comunes son:

- **Gestión de errores de API y red**

Si una solicitud a una API falla, el sistema puede reintentar la operación o mostrar un mensaje de error al usuario.

- **Validación de la entrada del usuario**

Si el usuario introduce datos inválidos, el sistema puede solicitar correcciones sin interrumpir la ejecución de la aplicación.

- **Prevención de fallos en la aplicación**

Al manejar valores inesperados o datos faltantes, se garantiza que el sistema continúe funcionando correctamente.

Se relacionan algunos ejemplos de manejo de errores en JavaScript, partiendo de lo siguiente:

¿Cómo manejar errores en código asíncrono? Cuando se trabaja con funciones asíncronas y promesas, los errores pueden gestionarse usando .catch().

```
fetch("https://jsonplaceholder.typicode.com/posts/1
")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error("Error fetching data:", error.message));
```

Si la solicitud falla (por ejemplo, por falta de conexión), el método .catch() maneja el error adecuadamente.

b) Manejo de errores con async y await

Para una gestión más clara en funciones asíncronas, se utiliza try...catch.

```
async function fetchData() {
try {
const response = await fetch("https://jsonplaceholder.typicode.com/posts/1
");
const data = await response.json();
console.log(data);
} catch (error) {
console.error("Error fetching data:", error.message);
}
}
fetchData();
```

Esto evita promesas no controladas y permite un manejo estructurado de errores.

c) Utilizar finally para limpieza

El bloque finally asegura que ciertas acciones se ejecuten siempre, incluso si ocurre un error.

```
function processTask() {
try {
console.log("Processing...");
throw new Error("Something went wrong!");
} catch (error) {
console.error("Caught error:", error.message);
} finally {
console.log("Task completed (cleaning up resources).");
}
}
processTask();
```

d) Diferentes tipos de errores de JavaScript

Los errores en JavaScript se clasifican en distintas categorías:

- **ReferenceError**

Se intenta usar una variable que no ha sido definida.

Ejemplo:

```
console.log(nonExistentVar); // ReferenceError
```

- **TypeError**

Se intenta ejecutar una operación sobre un tipo de dato incorrecto.

Ejemplo:

let num = 42;

num.toUpperCase(); // TypeError

- **SyntaxError**

Se presenta cuando la sintaxis del código no es válida.

Ejemplo:

javascript

CopyEdit

console.log("Hello; // SyntaxError: Falta cerrar una comilla

- **RangeError**

Ocurre cuando se excede el rango permitido de valores.

Ejemplo:

function recurse() {

return recurse();

}

recurse(); // RangeError: Se excedió el tamaño máximo de la pila de llamadas

5.3. Fallas de lógica

Las fallas de lógica en JavaScript son errores que no impiden que el programa se ejecute, pero sí producen resultados incorrectos, inconsistentes o distintos a los esperados. A diferencia de los errores de sintaxis, estas fallas no siempre muestran un mensaje en pantalla, por lo que suelen ser más difíciles de detectar. En la práctica profesional, este tipo de errores aparece cuando la lógica del programa, las condiciones, los cálculos o la manipulación de datos no reflejan correctamente la intención del desarrollador.

A continuación, se presentan algunas de las fallas lógicas más comunes en JavaScript, organizadas en distintos escenarios que suelen afectar el flujo del programa, la correcta evaluación de condiciones y el manejo de datos. Cada numeral ilustra un tipo de error frecuente, acompañado de ejemplos y correcciones, con el fin de ayudarte a identificar, comprender y prevenir estos problemas durante el desarrollo de aplicaciones:

a) Uso incorrecto de operadores

Una de las fallas más comunes en JavaScript ocurre al utilizar operadores equivocados dentro de una condición o una expresión. El caso más frecuente es usar el operador de asignación (=) en lugar de un operador de comparación (== o ===). Cuando esto sucede, el valor de la variable cambia y la condición puede evaluarse como verdadera, aunque esa no fuera la intención del programador. Este error altera el flujo lógico del programa y puede provocar comportamientos difíciles de identificar.

Ejemplo:

```
let a = 5;

if (a = 10) {
  console.log('Verdadero');
}

// Error: se usa '=' en lugar de '==' o '==='
```

En este caso, la variable a recibe el valor 10 y la condición resulta verdadera, por lo que el bloque se ejecuta. La forma correcta sería utilizar un operador de comparación estricta.

Corrección:

```
let a = 5;

if (a === 10) {
  console.log('Verdadero');
} else {
  console.log('Falso');
}
```

b) Condiciones mal planteadas

Las condiciones mal formuladas son otra fuente frecuente de fallas de lógica. Esto sucede cuando la expresión condicional no representa correctamente la regla del problema. Por ejemplo, al trabajar con edades, rangos, notas o límites, un pequeño

detalle como usar > en vez de >= puede modificar totalmente el resultado del programa.

Ejemplo:

```
let edad = 18;

if (edad > 18) {
  console.log('Mayor de edad');
}

// Error: debería ser >= 18
```

Aquí, una persona de 18 años no sería reconocida como mayor de edad, aunque en muchos contextos sí lo es. El error no está en la sintaxis, sino en la lógica de la condición.

Corrección:

```
let edad = 18;

if (edad >= 18) {
  console.log('Mayor de edad');
} else {
  console.log('Menor de edad');
}
```

c) Bucles infinitos

Un bucle infinito ocurre cuando la condición de repetición nunca deja de cumplirse o cuando la variable de control no se actualiza correctamente. Esta falla puede congelar la aplicación, consumir recursos innecesarios y afectar seriamente el rendimiento del sistema.

Ejemplo:

```
let i = 0;

while (i < 5) {

  console.log(i);

}

// Error: falta incrementar i
```

En este caso, la variable i nunca cambia, por lo que la condición i < 5 siempre será verdadera. El ciclo se ejecutará indefinidamente.

Corrección:

```
let i = 0;

while (i < 5) {

  console.log(i);

  i++;

}
```

d) Comparación de tipos incorrecta

JavaScript permite conversiones implícitas de tipos cuando se usa el operador ==. Esto puede generar resultados engañosos, ya que dos valores de diferente tipo pueden considerarse iguales. En aplicaciones reales, esta situación puede causar validaciones erróneas, decisiones equivocadas y errores de negocio.

Ejemplo:

```
console.log(5 == '5'); // true

console.log(5 === '5'); // false
```

Con ==, JavaScript convierte automáticamente la cadena '5' en número y luego realiza la comparación. Con ===, en cambio, también verifica el tipo de dato, lo cual lo hace más seguro.

Recomendación: usar preferiblemente el operador === para evitar ambigüedades y mantener un código más confiable.

e) Variables no inicializadas

Usar variables no inicializadas puede provocar resultados inesperados, como undefined o NaN. Este problema es común cuando el programador asume que una variable ya contiene un valor, pero en realidad nunca fue inicializada correctamente.

Ejemplo:

```
let x;

console.log(x + 5); // NaN
```

Como x no tiene un valor numérico asignado, la operación no puede realizarse adecuadamente y el resultado es NaN (Not a Number). Este tipo de fallas suele afectar cálculos, formularios y procesos de validación.

Corrección:

```
let x = 0;

console.log(x + 5); // 5
```

f) Índices fuera de rango

En los arreglos, acceder a una posición inexistente no produce necesariamente un error visible, pero sí devuelve undefined. Esto puede generar efectos en cadena cuando se intenta usar ese valor en operaciones, comparaciones o métodos posteriores.

Ejemplo:

```
let arr = [1, 2, 3];

console.log(arr[5]); // undefined
```

Si después se intenta sumar, convertir o manipular ese valor, el programa puede producir resultados erróneos. Por esta razón, es importante validar siempre los límites del arreglo.

Corrección:

```
let arr = [1, 2, 3];

if (arr[2] !== undefined) {

  console.log(arr[2]);
```

}

g) Mutación no deseada de datos

En JavaScript, cuando se asigna un arreglo u objeto a otra variable, en muchos casos no se crea una copia independiente, sino una referencia al mismo dato en memoria. Esto significa que, al modificar una variable, la otra también cambia. Esta situación genera fallas lógicas difíciles de detectar, especialmente en programas grandes.

Ejemplo:

```
let lista = [1, 2, 3];

let copia = lista;

copia[0] = 99;

// lista también cambia
```

Aquí, copia no es un nuevo arreglo, sino una referencia al mismo arreglo original. Por eso, al cambiar copia[0], también cambia lista[0].

Corrección:

```
let lista = [1, 2, 3];

let copia = [...lista];

copia[0] = 99;

// lista permanece igual
```

h) Retornos incorrectos en funciones

Otra falla lógica frecuente ocurre cuando una función no retorna el valor esperado o no retorna nada. En estos casos, el código puede seguir ejecutándose, pero las operaciones que dependen del resultado fallan.

Ejemplo:

```
function sumar(a, b) {
  let resultado = a + b;
}
console.log(sumar(2, 3)); // undefined
```

La función realiza el cálculo, pero no usa return. Por tanto, el resultado no sale de la función.

Corrección:

```
function sumar(a, b) {
  let resultado = a + b;
  return resultado;
}
console.log(sumar(2, 3)); // 5
```

i) Uso incorrecto de condiciones compuestas

Las condiciones compuestas con && y || pueden causar errores lógicos cuando no se agrupan correctamente o cuando se interpretan mal las prioridades de evaluación. Esto ocurre con frecuencia en validaciones complejas.

Ejemplo:

```
let edad = 20;

let tieneDocumento = false;

if (edad >= 18 || tieneDocumento) {

  console.log('Puede ingresar');

}
```

Si la regla real exige ser mayor de edad y además tener documento, esta condición está mal planteada, porque basta con cumplir una de las dos.

Corrección:

```
if (edad >= 18 && tieneDocumento) {

  console.log('Puede ingresar');

} else {

  console.log('No puede ingresar');

}
```

j) Confusión entre incremento previo y posterior

El uso de ++ antes o después de una variable puede cambiar el resultado de una expresión. Esta diferencia puede parecer pequeña, pero en ciclos, asignaciones y cálculos produce errores lógicos.

Ejemplo:

```
let x = 5;

let y = x++;

console.log(x); // 6

console.log(y); // 5
```

Aquí, y recibe primero el valor actual de x y luego x se incrementa. Si el desarrollador esperaba que y valiera 6, tendría una falla lógica.

Comparación:

```
let a = 5;

let b = ++a;

console.log(a); // 6

console.log(b); // 6
```

5.4. Manejo de errores y excepciones

Para entender estos errores, se presentan dos situaciones comunes en las que el manejo de errores resulta esencial para evitar fallos en la ejecución del programa: el

control de operaciones matemáticas inválidas y la gestión de datos incorrectos provenientes de fuentes externas. Estos ejemplos muestran cómo el uso de try...catch permite anticipar errores y mantener la estabilidad de la aplicación:

a) Manejo de la división por cero

A continuación, se tiene una función que divide un número entre otro. Para ello, se pasa parámetros a la función para ambos números. Deseando asegurarnos de que la división nunca genere un error al dividir un número entre cero (0).

Como medida preventiva, se ha escrito una condición que, si el divisor es cero, generará un error indicando que la división por cero no está permitida. En cualquier otro caso, se procederá con la división. Si se produce un error, el bloque catch lo gestionará y realizará las acciones necesarias (en este caso, registrar el error en la consola):

```
function divideNumbers(a, b) {
  try {
    if (b === 0) {
      const err = new Error("Division by zero is not allowed.");
      throw err;
    }
    const result = a/b;
    console.log(`The result is ${result}`);
  } catch(error) {
```

```
        console.error("Got a Math Error:", error.message)
    }
}
```

Ahora bien, si se invoca la función con los siguientes argumentos, se obtendrá un resultado de 5, y el segundo argumento es un valor distinto de cero:

```
divideNumbers(15, 3); // The result is 5
```

Pero si pasamos el valor 0 como segundo argumento, el programa generará un error y este se registrará en la consola.

```
divideNumbers(15, 0);
```

Producción:

b) Error matemático

A menudo, se recibirá JSON como respuesta a una llamada a la API. Se necesita analizar este JSON en el código JavaScript para extraer los valores. ¿Qué ocurre si la API te envía un JSON mal formado por error? No puede permitirse que la interfaz de usuario falle por esto. Se debe gestionar correctamente y aquí es donde el bloque try…catch vuelve a ser la salvación:

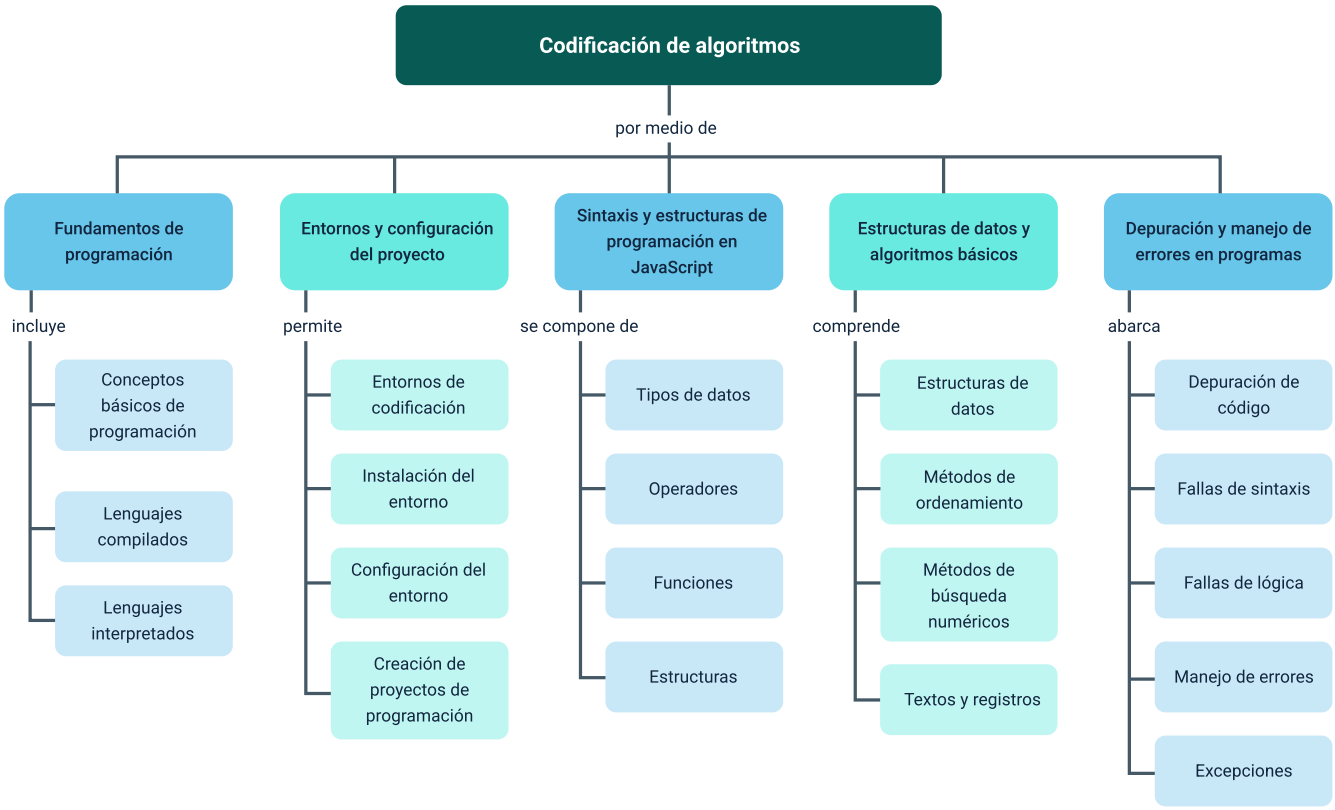
```
function parseJSONSafely(str) {
  try {
    return JSON.parse(str);
  } catch (err) {
```

```
    console.error("Invalid JSON:", err.message);
    return null;
  }
}
const userData = parseJSONSafely('{"name": "tapaScript"}'); // Parsed
const badData = parseJSONSafely('name: tapaScript');        // Handled gracefully
```

Sin try...catch, la segunda llamada provocará que la aplicación falle.

Síntesis

El componente formativo aborda de manera integral los fundamentos de la programación, iniciando con los conceptos básicos y la comprensión de los lenguajes compilados e interpretados. Posteriormente, se estudian los entornos de desarrollo, la instalación y configuración del ambiente de trabajo y la creación de proyectos, estableciendo una base sólida para la codificación. A continuación, se profundiza en la sintaxis y las estructuras de programación en JavaScript, incluyendo tipos de datos, operadores, funciones, estructuras de control, repetición y salto, que permiten construir soluciones lógicas y estructuradas. Asimismo, se introducen las estructuras de datos y los algoritmos básicos de ordenamiento y búsqueda, aplicados a diferentes tipos de información. Finalmente, el componente desarrolla habilidades para la depuración de código, la identificación de fallas de sintaxis y lógica, y el manejo de errores y excepciones, fortaleciendo la capacidad de crear programas confiables, eficientes y mantenibles.


Codificación de algoritmos
por medio de
Fundamentos de programación
incluye
Conceptos básicos de programación
Lenguajes compilados
Lenguajes interpretados
Entornos y configuración del proyecto
permite
Entornos de codificación
Instalación del entorno
Configuración del entorno
Creación de proyectos de programación
Sintaxis y estructuras de programación en JavaScript
se compone de
Tipos de datos
Operadores
Funciones
Estructuras
Estructuras de datos y algoritmos básicos
comprende
Estructuras de datos
Métodos de ordenamiento
Métodos de búsqueda numéricos
Textos y registros
Depuración y manejo de errores en programas
abarca
Depuración de código
Fallas de sintaxis
Fallas de lógica
Manejo de errores
Excepciones

Glosario

Algoritmo: es un conjunto ordenado y finito de pasos o instrucciones que se siguen para resolver un problema o realizar una tarea específica. En programación, los algoritmos son la base para desarrollar soluciones eficientes y estructuradas.

Bucle: es una estructura que permite repetir un bloque de instrucciones varias veces, ya sea un número determinado de veces o mientras se cumpla una condición.

Compilado: se refiere a un tipo de lenguaje que necesita ser traducido completamente a código máquina antes de ejecutarse. Este proceso se realiza mediante un compilador y suele generar programas más rápidos en ejecución.

Condicional: es una estructura de control que permite ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa, facilitando la toma de decisiones en el programa.

Expresión: es una combinación de valores, variables, operadores y funciones que se evalúa para producir un resultado dentro de un programa.

Función: es un bloque de código reutilizable que realiza una tarea específica. Puede recibir datos de entrada (parámetros), procesarlos y devolver un resultado, lo que permite estructurar mejor el programa.

Interpretado: es un tipo de lenguaje que se ejecuta línea por línea mediante un intérprete, sin necesidad de compilar todo el código previamente. Esto facilita la prueba y corrección de errores durante el desarrollo.

JavaScript: es un lenguaje de programación interpretado, orientado a objetos y basado en prototipos, ampliamente utilizado para desarrollar aplicaciones web interactivas, tanto en el lado del cliente como del servidor.

Lenguaje de programación: es un sistema formal compuesto por reglas y símbolos que permite a los desarrolladores comunicarse con la computadora para crear programas, aplicaciones y sistemas informáticos.

Matriz: es una estructura de datos bidimensional, es decir, un arreglo de arreglos, que permite organizar la información en filas y columnas para representar datos más complejos.

Objeto: es una estructura que permite almacenar información en forma de pares clave-valor, lo que facilita representar entidades del mundo real con múltiples atributos.

Operador: es un símbolo o conjunto de símbolos que permiten realizar operaciones sobre uno o más valores, como operaciones matemáticas, comparaciones o evaluaciones lógicas.

Variable: es un espacio reservado en la memoria del sistema que permite almacenar datos que pueden cambiar durante la ejecución de un programa. Las variables tienen un nombre identificador y un tipo de dato asociado.

Referencias bibliográficas

Divcode. (2023, 2 de enero). Cómo instalar JAVA en Visual Studio Code. [Video] YouTube. https://www.youtube.com/watch?v=57ekn6xnrqU

Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.). No Starch Press.

JAVA. (2025). ¿Cómo puedo descargar e instalar Java en un equipo con Windows de forma manual? https://www.java.com/es/download/help/windows_manual_download.html

JAVA. (2025). Instalación y uso de Oracle Java en macOS https://www.java.com/es/download/help/java_mac.html

Mozilla Developer Network (MDN). (2025). Referencia de JavaScript. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador `full stack`	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima